

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«КУБАНСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ»
(ФГБОУ ВО «КубГУ»)

Факультет компьютерных технологий и прикладной математики
Кафедра вычислительных технологий

ЛАБОРАТОРНАЯ РАБОТА №5
Дисциплина: «Методы поисковой оптимизации»

Работу выполнили: _____ Долгов М.И. и Костров А.А.

Направление подготовки: 02.03.02 Фундаментальная информатика и
информационные технологии

Преподаватель: _____ Нигодин Е.А.

Краснодар
2026

Тема. Пчелиный алгоритм

Цель. Реализовать приложение с графическим интерфейсом, демонстрирующее применение пчелиного алгоритма для решения задачи поиска минимума функции.

Алгоритм.

Пусть требуется найти минимум целевой функции:

$$f(\mathbf{x}) \rightarrow \min, \mathbf{x} \in D \subset \mathbb{R}^n$$

где:

- n — размерность пространства поиска;
- D — допустимая область, задаваемая ограничениями:
- $D = \{\mathbf{x} \mid x_{\min}^i \leq x_i \leq x_{\max}^i, i = 1, \dots, n\}$

В алгоритме используется множество пчёл роя S , где:

$S = S^o \cup S^w$, S^o – множество пчёл разведчиков, S^w – множество остальных пчёл

$$S^{\{o\}} = \{s_i^{\{o\}}, i \in [1: |S^{\{o\}}|]\}$$

$$S^{\{w\}} = \{s_{\{i\}}^{\{w\}}, i \in [1: |S^{\{w\}}|]\} = \{s_{\{i\}}, i \in [|S^{\{o\}}| + 1: |S|]\}$$

Пчёлы-разведчики находят наиболее оптимальные точки для расположения элитных $A^{\{b\}}$ и лучших спотов на области D с помощью значениям фитнес-функции, соответствующей расположению каждой пчелы из популяции

$$A^{\{b\}} = \{a_{\{j\}}^{\{b\}}, j \in [1: |A^{\{b\}}|]\}$$

$$A^{\{p\}} = \{a_{\{k\}}^{\{p\}}, p \in [1: |A^{\{p\}}|]\}$$

При инициализации пчёл-разведчиков, если их точки не удовлетворяют неравенству вида:

$$\|X_{k_1}^p - X_{k_2}^p\|_E \leq \varepsilon_p, k_1, k_2 \in [1: |A^p|], k_1 \neq k_2,$$

Берётся новая точка посередине отрезка, проведённому между точками, оба разведчика исключаются

Структура алгоритма:

- 1) Сгенерировать случайные точки X_i , отправить разведчиков, вычислить фитнес $\varphi(X_i)$, отсортировать по убыванию.
- 2) Создать новых разведчиков, выбрать лучшие точки: первые — центры элитных участков, следующие — перспективных.
- 3) Вокруг выбранных центров отправить фуражиров (рабочих пчёл) (больше — в элитные), случайно в окрестностях, вычислить фитнес.
- 4) Проверить критерий останова; если нет — объединить все точки, отсортировать и повторить с шага 2.

Основные параметры алгоритма

$|S^o|$ – число пчел-разведчиков;

$|A^b|$ – число элитных участков;

$|A^p|$ – число перспективных участков;

n_b – число пчел, отправляемых на элитные участки;

n_p – число пчел, отправляемых на перспективные участки;

$r_i^b, r_i^p, i \in [1: |X|]$ – радиусы участков.

Общее число пчёл равно

$$|S^w| = n_b |A^b| + n_p |A^p| = 7 \cdot 2 + 4 \cdot 3 = 26.$$

В качестве критерия останова используется достижение фиксированного количества итераций или удовлетворение коэффициенту стагнации:

1. Программа представляет собой desktop-приложение, реализованное на языке Python с использованием фреймворка PySide6 и библиотеки pyqtgraph для визуализации. Приложение предназначено для демонстрации алгоритмов оптимизации функций в 3D-пространстве (рисунок 1).

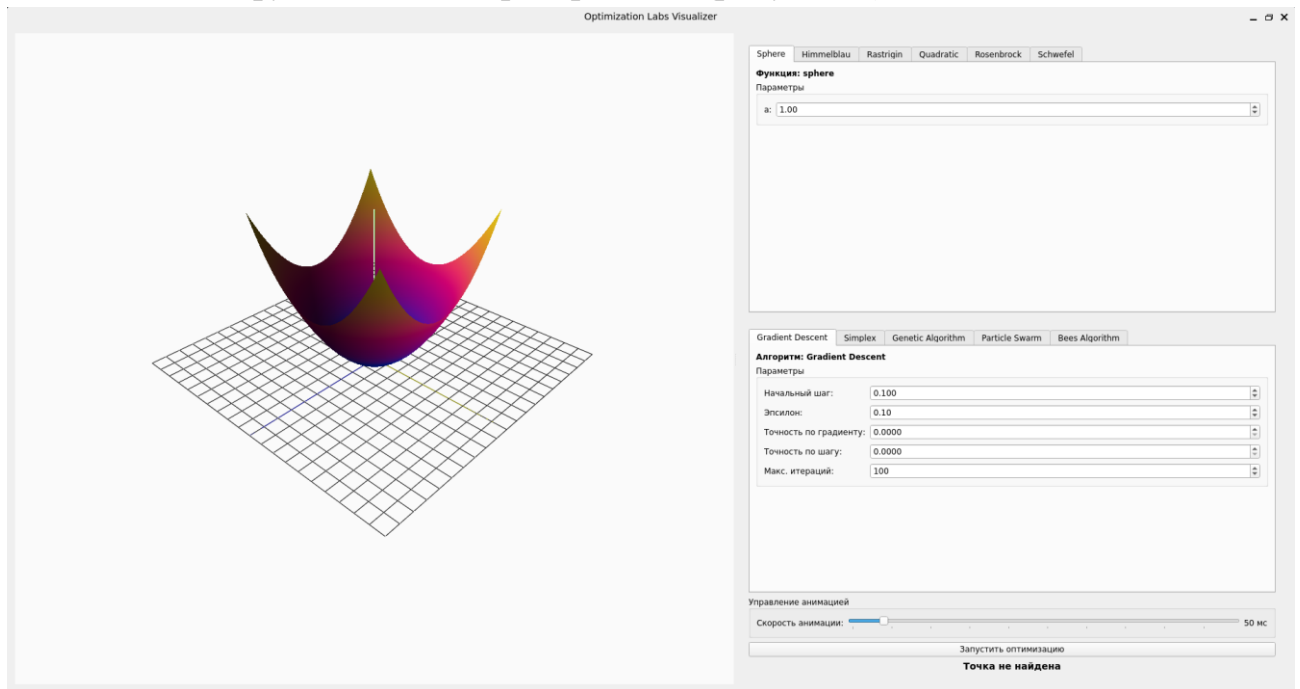


Рисунок 1 — Главное окно приложения

2. Доступные функции: Sphere, Himmelblau, Rastrigin, Quadratic и Schwefel (рисунок 2).

Sphere Himmelblau Rastrigin Quadratic Rosenbrock Schwefel

Функция: rastrigin

Параметры

a: 10.00

Рисунок 2 — Доступные функции

3. Доступные алгоритмы оптимизации: градиентный спуск с постоянным шагом, симплекс-метод, генетический алгоритм, алгоритм роя частиц, пчелиный алгоритм (рисунок 3).

Gradient Descent Simplex Genetic Algorithm Particle Swarm Bees Algorithm

Алгоритм: Bees Algorithm

Параметры

Число разведчиков: 16

Элитные участки: 2

Лучшие участки: 3

Пчёлы в элитных участках: 7

Пчёлы в лучших участках: 4

Радиус элитных участков: 0.40

Радиус лучших участков: 0.20

Макс. итераций: 500

Порог стагнации: 20

Порог улучшения (ϵ): 2.00

Управление анимацией

Скорость анимации: 50 мс

Запустить оптимизацию

Рисунок 3 — Доступные алгоритмы оптимизации

4. Запустим алгоритм поиска минимума функции Rastrigin (рисунок 4).

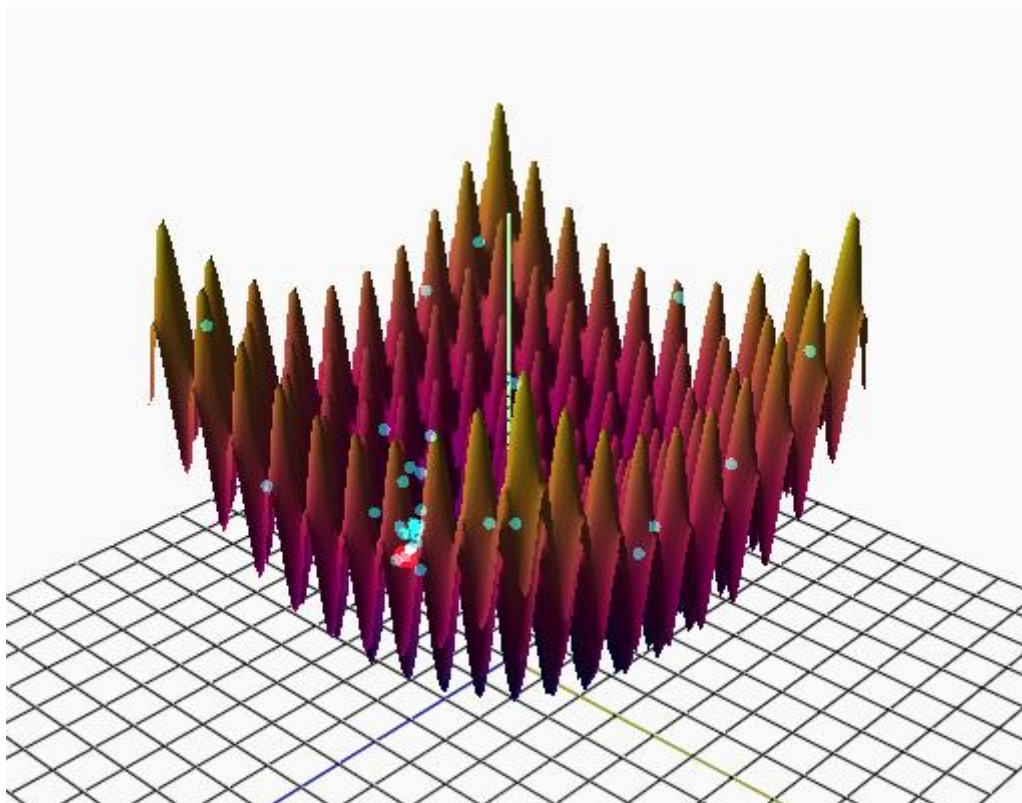


Рисунок 4 — Запуск алгоритма оптимизации

5. Завершим пчелиный алгоритм поиска минимума функции Rastrigin (рисунок 5).

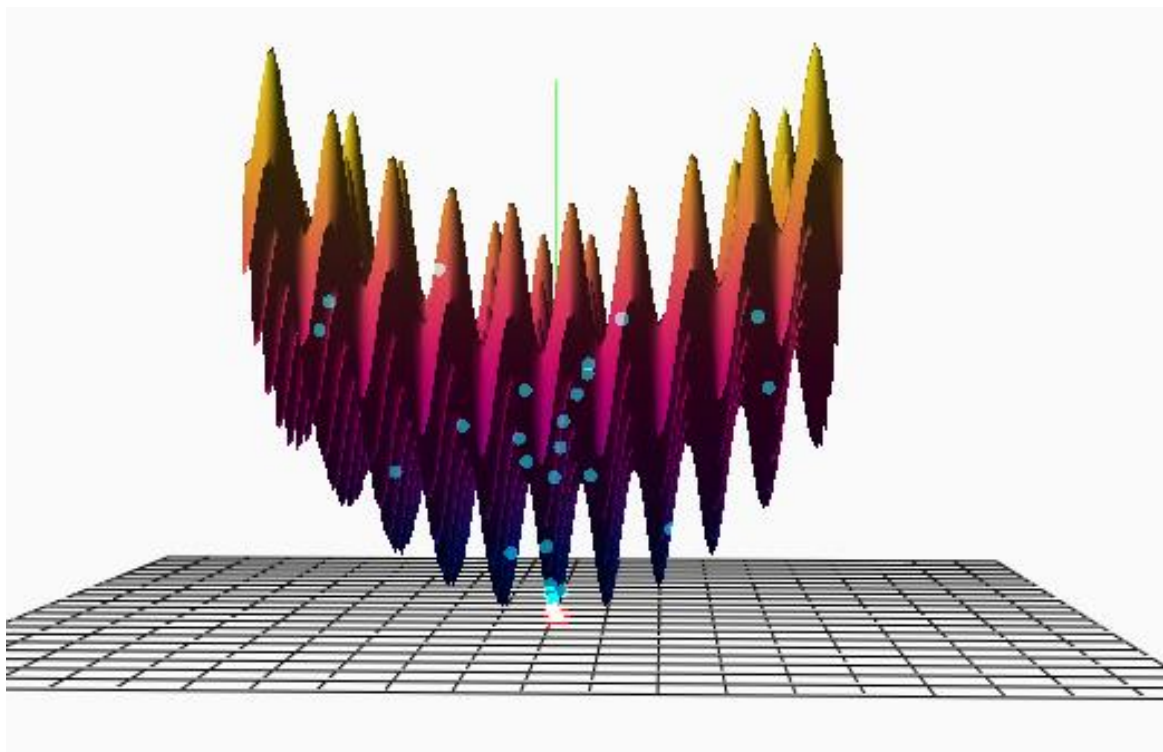


Рисунок 5 — Запуск алгоритма

Вывод. В ходе данной лабораторной работы было реализовано приложение с графическим интерфейсом, демонстрирующее применение пчелиного алгоритма для решения задачи поиска минимума функции.

Листинг программы.

```
import numpy as np
from typing import Generator, List

class BeesAlgorithm:
    """Пчёлковый алгос"""
    def __init__(
        self,
        func,
        func_params,
        bounds: list[tuple],
        n_scouts: int = 16,
        n_elite: int = 2,
        n_best: int = 3,
        n_elite_bees: int = 7,
        n_best_bees: int = 4,
        r_elite: float = 0.2,
        r_best: float = 0.4,
        max_iter: int = 500,
        stagnation: int = 20,
        eps: float = 1.0,
    ):
        self.func = func
        self.func_params = func_params
        self.bounds = np.array(bounds, dtype=float)
        self.dim = len(bounds)
        self.n_scouts = n_scouts
        self.n_elite = n_elite
        self.n_best = n_best
        self.n_elite_bees = n_elite_bees
        self.n_best_bees = n_best_bees
        self.r_elite = r_elite
        self.r_best = r_best
        self.max_iter = max_iter
        self.stagnation = stagnation
        self.spot_eps = eps
        self.stagnation_param = 3

    def _random_point(self) -> np.ndarray:
        lo, hi = self.bounds[:, 0], self.bounds[:, 1]
        return lo + np.random.rand(self.dim) * (hi - lo)

    def _fitness(self, p: np.ndarray):
        return self.func(*p, **self.func_params)
```

```

def _random_worker_point_in_spot_area(self, spot_p, r) -> np.ndarray:
    delta = np.random.uniform(-r, r, size=self.dim)
    p = np.clip(spot_p + delta, self.bounds[:, 0], self.bounds[:, 1])
    return float(p[0]), float(p[1])

@staticmethod
def _euclidean_distance(p1, p2):
    return np.linalg.norm(p1 - p2)

@staticmethod
def _midpoint(p1, p2):
    return (p1 + p2) / 2

def _spawn_scouts(self, n_scouts: int) -> list[np.ndarray]:
    lo, hi = self.bounds[:, 0], self.bounds[:, 1]
    selected = []

    while len(selected) < n_scouts:
        # генерируем пачку кандидатов сразу
        batch = lo + np.random.rand(max(n_scouts * 4, 64), self.dim) * (hi -
lo)

        for candidate in batch:
            if not selected:
                selected.append(candidate.astype(np.float64))
            else:
                # расстояния до всех принятых — одна матричная операция
                dists = np.linalg.norm(np.array(selected) - candidate, axis=1)
                if dists.min() >= self.spot_eps:
                    selected.append(candidate.astype(np.float64))

        if len(selected) == n_scouts:
            return selected

    return selected

def run(self) -> Generator[List[np.float64], None, None]:
    # 1) scouts
    stagnation_cnt = 0
    iter = 0
    best_history: list[float] = []

    new_population = [(self._fitness(p), p) for p in
self._spawn_scouts(self.n_scouts)]
    new_population.sort(key=lambda t: t[0])
    population = new_population
    yield [x[1] for x in population]

    progress = 0
    while stagnation_cnt < self.stagnation and iter < 500:
        curr_scouts = max(1, int(self.n_scouts * (1-progress)))

```



```

curr_elite = max(1, int(self.n_elite * (1 - progress)))
curr_best = max(0, int(self.n_best * (1 - progress)))
curr_r_elite = max(self.r_elite * (1 - progress), 1e-6)
curr_r_best = max(self.r_best * (1 - progress), 1e-6)

iter += 1
progress = iter / 500

# 2) zones
elite_centers = [x for _, x in population[:curr_elite]]
best_centers = [x for _, x in population[curr_elite:curr_elite +
curr_best]]
new_population = population[:curr_elite+curr_best] + \
    [(self._fitness(p), p) for p in self._spawn_scouts(curr_scouts)]

# 3) send workers
for center in elite_centers:
    for _ in range(self.n_elite_bees):
        p = self._random_worker_point_in_spot_area(center,
curr_r_elite)
        new_population.append((self._fitness(p), p))

for center in best_centers:
    for _ in range(self.n_best_bees):
        p = self._random_worker_point_in_spot_area(center, curr_r_best)
        new_population.append((self._fitness(p), p))

population = new_population
population.sort(key=lambda t: t[0])
best_history.append(population[0][0])
dots = [x[1] for x in population][::-1]
yield dots

# 4) stagnation
if len(best_history) > 10:
    if abs(best_history[-1] - best_history[-self.stagnation_param]) <
1e-6:
        stagnation_cnt += 1
    else:
        stagnation_cnt = 0

def bees_algorithm(
    func,
    x_bounds: tuple,
    y_bounds: tuple,
    n_scouts: int = 16,
    n_elite: int = 2,
    n_best: int = 3,
    n_elite_bees: int = 7,
    n_best_bees: int = 4,

```

```

    r_elite:        float = 0.4,
    r_best:         float = 0.2,
    max_iter:       int   = 500,
    stagnation:     int   = 20,
    eps:            float = 2.0,
    clear_view=None,
    **func_params
) -> Generator:
    alg = BeesAlgorithm(
        func=func, func_params=func_params, bounds=[x_bounds, y_bounds],
        n_scouts=n_scouts, n_elite=n_elite, n_best=n_best,
        n_elite_bees=n_elite_bees, n_best_bees=n_best_bees,
        r_elite=r_elite, r_best=r_best,
        max_iter=max_iter, stagnation=stagnation, eps=eps,
    )
    for solution in alg.run():
        if clear_view:
            clear_view()
        yield solution

```